NumPy Cheat Sheet

Covers: Fundamentals → Advanced Indexing & Broadcasting → Useful Functions

---

1. What is NumPy?

NumPy (Numerical Python) is the core library for scientific computing in Python. Its main object is the `ndarray` — an n-dimensional array of homogeneous data — plus fast routines for math, linear algebra, statistics, sorting, and more.

NumPy arrays vs Python lists

	Python list	**NumPy array**
Size	Dynamic (can grow)	Fixed at creation — resizing makes a new array
Data types	Mixed types allowed	Single dtype, same size in memory
Speed	Slower (loop-based)	Much faster (vectorized, ~50x in tests)
Memory	Higher overhead	Compact, contiguous storage

```
import numpy as np
import time

a = [i for i in range(10_000_000)]
b = [i for i in range(10_000_000, 20_000_000)]
c = []
start = time.time()
for i in range(len(a)):
    c.append(a[i] + b[i])
print(time.time() - start)        # e.g. ~3.2s

a, b = np.arange(10_000_000), np.arange(10_000_000, 20_000_000)
start = time.time()
c = a + b
print(time.time() - start)        # e.g. ~0.06s — ~50x faster
```

Memory comparison: `sys.getsizeof(python_list)` vs `sys.getsizeof(np.arange(n, dtype=np.int8))` — the NumPy array uses dramatically less memory because every element is a fixed-size, tightly packed `int8` instead of a full Python object.

---

2. Creating Arrays

```
import numpy as np

np.array([1, 2, 3])                         # 1D
np.array([[1, 2, 3], [4, 5, 6]])            # 2D
np.array([[[1,2],[3,4]],[[5,6],[7,8]]])     # 3D
np.array([1, 2, 3], dtype=float)            # set dtype

np.arange(1, 11, 2)                         # [1 3 5 7 9]
np.arange(16).reshape(2, 2, 2, 2)           # range + reshape

np.ones((3, 4))                             # all 1s
np.zeros((3, 4))                            # all 0s
np.random.random((3, 4))                    # uniform [0,1)
np.linspace(-10, 10, 10, dtype=int)         # 10 evenly spaced points
np.identity(3)                              # identity matrix
```

---

3. Array Attributes

```
a1 = np.arange(10, dtype=np.int32)
a2 = np.arange(12, dtype=float).reshape(3, 4)
a3 = np.arange(8).reshape(2, 2, 2)

a3.ndim       # number of dimensions -> 3
a3.shape      # shape tuple -> (2, 2, 2)
a2.size       # total element count -> 12
a3.itemsize   # bytes per element
a1.dtype      # data type of elements
```

Changing dtype

```
a3.astype(np.int32)   # returns a copy with new dtype
```

---

4. Array Operations

```
a1 = np.arange(12).reshape(3, 4)
a2 = np.arange(12, 24).reshape(3, 4)

# Scalar operations
a1 ** 2          # arithmetic — applies to every element
a2 == 15         # relational — elementwise boolean array

# Vector operations (elementwise between two arrays of same shape)
a1 ** a2
```

---

5. Array Functions

```
a1 = np.round(np.random.random((3, 3)) * 100)

# Aggregations — axis=0 is column-wise, axis=1 is row-wise
np.prod(a1, axis=0)    # also: np.max, np.min, np.sum
np.var(a1, axis=1)     # also: np.mean, np.median, np.std

np.sin(a1)             # trig functions: sin, cos, tan, etc.
np.exp(a1)             # exponent (and np.log for logarithm)
np.ceil(np.random.random((2, 3)) * 100)   # also: np.floor, np.round

# Dot product (matrix multiplication)
a2 = np.arange(12).reshape(3, 4)
a3 = np.arange(12, 24).reshape(4, 3)
np.dot(a2, a3)         # (3,4) · (4,3) -> (3,3)
```

---

6. Indexing and Slicing

```
a1 = np.arange(10)
a2 = np.arange(12).reshape(3, 4)
a3 = np.arange(8).reshape(2, 2, 2)

a2[1, 0]        # single element, row 1 col 0
a3[1, 0, 1]     # 3D indexing
a3[1, 1, 0]

a1[2:5:2]       # slice with step
a2[0:2, 1::2]   # rows 0-1, every other col starting at 1
a2[::2, 1::2]   # every other row, every other col from 1
a2[1, ::3]      # row 1, every 3rd element
a2[0, :]        # entire row 0
a2[:, 2]        # entire column 2
a2[1:, 1:3]     # sub-matrix

a3 = np.arange(27).reshape(3, 3, 3)
a3[::2, 0, ::2]
a3[2, 1:, 1:]
a3[0, 1, :]
```

---

7. Iterating

```
for i in a1:          # 1D — iterates over elements
    print(i)

for i in a2:           # 2D — iterates over rows
    print(i)

for i in a3:           # 3D — iterates over 2D sub-arrays
    print(i)

for i in np.nditer(a3):   # flattens completely — iterates every scalar
    print(i)
```

---

8. Reshaping, Transpose & Ravel

```
a2.reshape(4, 3)     # change shape (data unchanged, total size must
match)

np.transpose(a2)     # swap axes
a2.T                 # shorthand for transpose

a3.ravel()           # flatten to 1D (returns a view when possible)
```

---

9. Stacking and Splitting

```
a4 = np.arange(12).reshape(3, 4)
a5 = np.arange(12, 24).reshape(3, 4)

np.hstack((a4, a5))   # horizontal stack -> side by side (more columns)
np.vstack((a4, a5))   # vertical stack -> stacked rows (more rows)

np.hsplit(a4, 5)       # split into 5 equal column-wise pieces
np.vsplit(a5, 2)       # split into 2 equal row-wise pieces
```

---

10. Advanced Indexing

```
a = np.arange(24).reshape(6, 4)

# Normal indexing/slicing
a[1, 2]
a[1:3, 1:3]

# Fancy indexing — pass a list of indices
a[:, [0, 2, 3]]        # select specific columns

# Boolean indexing — pass a condition, get matching elements
a = np.random.randint(1, 100, 24).reshape(6, 4)
a[a > 50]                       # all values > 50
a[a % 2 == 0]                   # all even values
a[(a > 50) & (a % 2 == 0)]      # combine conditions with & / |
a[~(a % 7 == 0)]                # negate a condition with ~
```

---

11. Broadcasting

Broadcasting lets NumPy perform arithmetic on arrays of **different shapes** by "stretching" the smaller one without copying data.

Rules

1. If two arrays have a different number of dimensions, prepend size-1 dimensions to the smaller one until both have the same number of dimensions.
2. For each dimension, the sizes must either match, or one of them must be 1 (that dimension is then stretched to match the other).
3. If neither condition holds for some dimension, broadcasting fails and NumPy raises an error.

```
# Same shape — straightforward elementwise add
a = np.arange(6).reshape(2, 3)
b = np.arange(6, 12).reshape(2, 3)
a + b

# Different shape — b is broadcast across a
a = np.arange(6).reshape(2, 3)
b = np.arange(3).reshape(1, 3)
a + b          # b's single row is applied to every row of a

# (1,3) + (3,1) -> broadcasts to (3,3)
a = np.arange(3).reshape(1, 3)
b = np.arange(3).reshape(3, 1)
a + b

# (1,3) + (4,1) -> broadcasts to (4,3)
a = np.arange(3).reshape(1, 3)
b = np.arange(4).reshape(4, 1)
a + b
```

```
# Incompatible shapes -> ValueError
a = np.arange(12).reshape(3, 4)
b = np.arange(12).reshape(4, 3)
a + b   # fails: dimensions (3,4) vs (4,3) don't match and neither is 1
```

---

12. Mathematical Formulas with NumPy

```
# Sigmoid activation
def sigmoid(array):
    return 1 / (1 + np.exp(-array))

# Mean Squared Error
def mse(actual, predicted):
    return np.mean((actual - predicted) ** 2)
```

13. Working with Missing Values

```
a = np.array([1, 2, 3, 4, np.nan, 6])

np.isnan(a)        # boolean mask of NaNs
a[~np.isnan(a)]    # filter out NaNs
```

14. Plotting with Matplotlib

```
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)
plt.plot(x, x)              # y = x
plt.plot(x, x ** 2)         # y = x^2
plt.plot(x, np.sin(x))      # y = sin(x)
plt.plot(x, 1 / (1 + np.exp(-x)))   # sigmoid curve
```

15. Meshgrids

`np.meshgrid` builds coordinate matrices from coordinate vectors — used heavily for evaluating functions over a 2D grid (e.g. for contour/3D plots).

```
x = np.linspace(-4, 4, 9)
y = np.linspace(-5, 5, 11)

xx, yy = np.meshgrid(x, y)   # xx and yy are 2D grids of x- and y-
coordinates
z = xx ** 2 + yy ** 2        # evaluate a function over the grid

# plt.contourf(xx, yy, z)
```

16. Useful NumPy Functions Reference

`np.sort` — sorted copy of an array

```
np.sort(a)            # ascending
np.sort(a)[::-1]      # descending
np.sort(b, axis=0)    # sort each column independently
```

`np.append` — add values to the end of an array

```
np.append(a, 200)
np.append(b, np.random.random((b.shape[0], 1)), axis=1)   # add a column
```

`np.concatenate` — join arrays along an existing axis

```
np.concatenate((c, d), axis=0)   # stack along rows
np.concatenate((c, d), axis=1)   # stack along columns
```

`np.unique` — distinct values in an array

```
np.unique(np.array([1,1,2,2,3,3]))   # -> [1 2 3]
```

`np.expand_dims` — insert a new axis

```
np.expand_dims(a, axis=0).shape   # add a dimension at position 0
np.expand_dims(a, axis=1)         # turn 1D into column vector
```

`np.where` — conditional selection / replacement

```
np.where(a > 50)            # indices where condition is True
np.where(a > 50, 0, a)      # replace matches with 0, keep rest
np.where(a % 2 == 0, 0, a)  # zero-out even numbers
```

`np.argmax` / `np.argmin` — index of max/min value

```
np.argmax(a)             # index of the largest element (flattened)
np.argmax(b, axis=0)      # per-column
np.argmax(b, axis=1)      # per-row
np.argmin(a)
```

`np.cumsum` / `np.cumprod` — running totals

```
np.cumsum(a)             # cumulative sum, flattened
np.cumsum(b, axis=1)     # cumulative sum across each row
np.cumprod(a)            # cumulative product
```

`np.percentile` / `np.median`

```
np.percentile(a, 50)     # 50th percentile
np.median(a)             # same as 50th percentile
```

`np.histogram` — frequency distribution

```
np.histogram(a, bins=[0, 50, 100])   # counts per bin range
```

`np.corrcoef` — Pearson correlation

```
salary = np.array([20000, 40000, 25000, 35000, 60000])
experience = np.array([1, 3, 2, 4, 2])
np.corrcoef(salary, experience)   # correlation matrix
```

`np.isin` — membership test

```
items = [10, 20, 30, 40, 50]
a[np.isin(a, items)]     # keep only elements that appear in items
```

`np.flip` — reverse order along an axis

```
np.flip(a)            # reverse a 1D array
np.flip(b, axis=1)    # reverse each row
```

`np.put` — replace elements in place (flattened indices)

```
np.put(a, [0, 1], [110, 530])   # set a.flat[0]=110, a.flat[1]=530
```

`np.delete` — remove elements/sub-arrays

```
np.delete(a, [0, 2, 4])   # remove elements at these (flattened) indices
```

Set functions

```
m = np.array([1, 2, 3, 4, 5])
n = np.array([3, 4, 5, 6, 7])

np.union1d(m, n)        # all unique values from both -> [1 2 3 4 5 6 7]
np.intersect1d(m, n)    # values in both -> [3 4 5]
np.setdiff1d(n, m)      # in n but not in m -> [6 7]
np.setxor1d(m, n)       # in exactly one of the two -> [1 2 6 7]
m[np.in1d(m, 1)]        # boolean mask: is each element of m in [1]?
```

`np.clip` — limit values to a range

```
np.clip(a, a_min=25, a_max=75)   # values below 25 -> 25, above 75 -> 75
```

`np.swapaxes` — swap two axes of an array

```
a3 = np.arange(8).reshape(2, 2, 2)
np.swapaxes(a3, 0, 2)   # swap axis 0 and axis 2
```

`np.random.uniform` — random floats over a custom range

```
np.random.uniform(low=0, high=10, size=(3, 3))   # uniform values in [0, 10)
```

`np.count_nonzero` — count non-zero (or matching) elements

```
np.count_nonzero(a)              # count of nonzero elements
np.count_nonzero(a > 50)         # count of elements satisfying a condition
```

`np.tile` — repeat an entire array as a tiled pattern

```
np.tile(np.array([1, 2, 3]), 2)        # -> [1 2 3 1 2 3]
np.tile(np.array([[1, 2]]), (2, 3))     # tile a 2D block 2x down, 3x across
```

`np.repeat` — repeat each element individually

```
np.repeat(np.array([1, 2, 3]), 2)       # -> [1 1 2 2 3 3]
np.repeat(np.array([1, 2, 3]), [1, 2, 3])   # different repeat count per element
```

`np.allclose` / `np.array_equal`

```
np.allclose(a, b)        # True if all elements are approximately equal (float-safe)
np.array_equal(a, b)     # True only if shapes and all values match exactly
```

Quick Reference: Axis Convention

	`axis=0`	`axis=1`
Effect	Operates **down columns**	Operates **across rows**

Result shape	One value per column	One value per row

Source notebooks: Session 13 (Fundamentals), Session 14 (Advanced), Session 15 (Tricks & Functions)